BEHAVIORAL STUDY · v1 · 2026

bonereaper

Behavioral Study

AI-led decomposition of one of Polymarket's top 5-min BTC wallets.
Output is features, weights, and a research dashboard — not orders.

TARGET WALLET	PROFILE	OUTPUT	STAGE
0xeebd...eba30	polymarket.com/@bonereaper	labelled features + dashboard	pre-Phase A

Table of Contents

A focused research-stream master plan

01	**Executive Summary**	What we're studying and why.	p. 3
02	**Why Study, Not Copy**	Six failure modes of direct copy-trading.	p. 4
03	**Target Profile**	bonereaper wallet identity, observable signals.	p. 5
04	**System Architecture**	Data pipeline diagram end-to-end.	p. 6
05	**AI Agent Roles**	3 offline analyzers; their inputs/outputs/cost.	p. 7
06	**Dashboard Layout**	7-tab Streamlit research console.	p. 8
07	**Phase A — Pre-flight**	Validate target is worth studying. Abort gate.	p. 9
08	**Phase B — Data Ingestion**	Pull on-chain trade history.	p. 10
09	**Phase C — State Reconstruction**	Rebuild market context per trade.	p. 11
10	**Phase D — Feature Engineering**	Build labelled dataset.	p. 12
11	**Phase E — Statistical Analysis**	Fingerprint, importance, disagreement.	p. 13
12	**Phase F — AI-Narrated Insights**	Run the 3 analysis agents.	p. 14
13	**Phase G — Dashboard Build**	Streamlit MVP first; stretch tabs gated.	p. 15
14	**Phase H — Bot Integration**	Feed weights into Phase 3 backtest.	p. 16
15	**Phase I — Decay & Adversarial**	Daily monitor, quarterly check.	p. 17
16	**Output Schemas**	What we ship from this study.	p. 18
17	**Risk Register**	Study-specific risks.	p. 19
18	**Cost & Timeline**	Cumulative spend per phase.	p. 20
19	**Operating Discipline**	Anti-vibe rules apply here too.	p. 21
20	**Sources**	On-chain + Polymarket + ML references.	p. 22

Executive Summary

What this document is

This is a focused research-stream master plan: an AI-led behavioral decomposition of **bonereaper** (Polymarket wallet `0xeebde7a0e019a63e6b476eb425505b7b3e6eba30`), one of the most profitable wallets on the 5-minute BTC up/down markets. We do not copy-trade. We treat each of their on-chain trades as a high-confidence label for the market state at decision time, learn what features predict their entries, and feed the resulting weights into our main bot's strategy.

≥60%	**3**	**7**	**$10-50**
Pre-flight win-rate gate (else abort study)	Offline analysis agents (Anthropic API)	Streamlit dashboard tabs (3 MVP + 4 stretch)	One-time analysis cost; ~$1/wk ongoing

Output of this study:

- **Labelled dataset** at `state/research/bonereaper/labelled.parquet` — every observable trade with reconstructed market state and AI-narrated rationale.
- **Feature weights** at `weights/<date>.yaml` — versioned, A/B'd against current weights before promotion.
- **Streamlit research dashboard** — localhost only, indexes the labelled dataset live.
- **Edge-decay monitor** — daily cron, Telegram-alerted when bonereaper's edge weakens.
- **Adversarial-resistance report** — quarterly, checks for bait-shaped market correlation.

What we will NEVER build: a real-time copy-trade execution path, a replay-with-1-second-lag bot, a "buy what they buy" service. This study is off the live trading critical path entirely.

Why Study, Not Copy

Six reasons direct copy-trading fails

01 **Confirmation lag**
On Polygon, on-chain confirmation is 1-2s. On a 300s market, by the time you observe their trade, the orderbook has already absorbed it and the edge is gone. Copy-trade returns approach zero as the source approaches sub-second decision latency.

02 **Sample-of-one risk**
If bonereaper blows up next month — strategy decay, market regime shift, smart contract bug — your strategy dies with theirs. You have no diagnostic, no recovery, no alternative.

03 **No understanding**
When their behavior shifts (and it will), you can't tell why. You can't decide whether to keep mimicking, demote signal weight, or stop. You're reading their actions but not their reasoning.

04 **Adversarial trapping**
If a profitable wallet detects copiers, they can place a string of small loss-leader trades to bait copiers into bad positions, then reverse course. Public on-chain wallets are observable by anyone — including the wallet's own operators.

05 **Survivorship bias**
bonereaper may be the visible winner of a multi-wallet portfolio — the operator runs N wallets, and we only see the lucky one. Copying any single wallet is biased toward exactly the strategies that look successful in retrospect.

06 **Cost-basis mismatch**
They may have maker rebates, fee discounts, or co-located infrastructure that materially changes their breakeven point. The same trade for them at $0.94 fill might be profitable; for you at $0.96 fill, it isn't.

The reframe: Each bonereaper trade is a high-confidence *label*. We learn what *features* of the market state predict their entries, and use those features in our own strategy. Output is feature weights, not copy-orders.

Target Profile

What we know about bonereaper before we start

This page is what we know *now*. Phase A pulls actual on-chain numbers and either confirms or kills the study.

Identifiers

Field	Value
Wallet address (Polygon)	`0xeebde7a0e019a63e6b476eb425505b7b3e6eba30`
Polymarket profile	polymarket.com/@bonereaper
Markets traded	Polymarket BTC 5-min up/down (slug pattern: btc-updown-5m-{window_ts})
Settlement chain	Polygon mainnet (chainId 137)
Settlement asset	USDC.e (migration to native USDC tracked separately)

Observable signals (public on-chain)

Signal	How we read it
Trade entries	Polymarket exchange-contract events filtered by wallet address; events expose market, side, size, fill price.
Entry timing	Block timestamp + transaction order; estimate decision time as block_ts – latency budget (typically T-15s to T-5s relative to window close).
Position outcomes	Resolution event from Polymarket UMA oracle proposal once the window closes; pair with entry to compute per-trade P&L.;
Bankroll trajectory	USDC balance over time on the wallet; growth rate and volatility characterize their risk regime.
Behavior shifts	Changes in average size, frequency, time-of-day distribution, or UP/DOWN balance signal a strategy change — log them as eras.

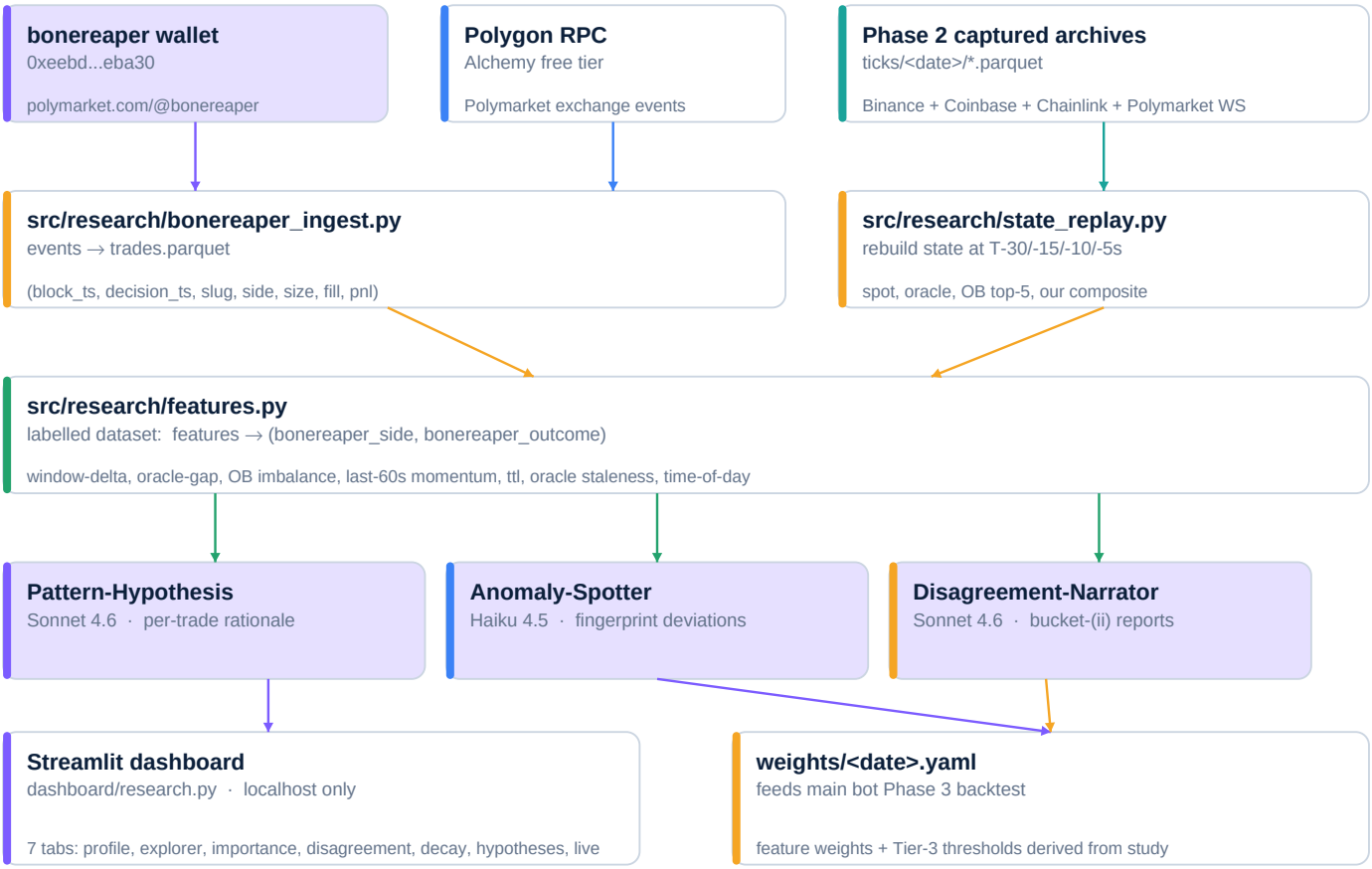
Pre-flight unknowns (Phase A resolves):

- 30/90-day rolling win-rate after fees.
- Total trade count over observable window.
- Average size + size distribution.
- UP/DOWN balance.
- Time-of-day entry heatmap.
- Behavior-shift dates (visible regime changes).

System Architecture

End-to-end data pipeline

Three input sources flow into a four-stage offline pipeline. No live trading code path. Output drops into the main bot's Phase 3 backtest as data, never as orders.



All offline / batched. No live trading code path. Output is features and reports, never orders.

AI Agent Roles

Three offline analyzers — Anthropic API, batched, cached

All three agents run offline against the labelled dataset. None are on the live trading hot path. Total one-time cost ~$10-50 with prompt caching; <$5/week ongoing as new bonereaper trades land.

Pattern-Hypothesis (Sonnet 4.6)

Input	Reconstructed market state at T-30/-15/-10/-5s plus bonereaper's choice.
Output	1-paragraph natural-language hypothesis for *why* this trade — written into a `rationale` field on each labelled record.
Why useful	Surfaces patterns simple feature extraction misses (e.g. 'oracle is 47s stale; spot has rallied 0.4% in the last 30s; orderbook is selling off — they're betting on the resolution catching up to spot').
When run	Once per trade, batched. Re-run for new trades on a daily cron.
Cost est.	~$0.005/trade with cache. 5k trades = $25 one-time.

Anomaly-Spotter (Haiku 4.5)

Input	Trade where bonereaper deviates from their typical fingerprint (oversize, off-hour, UP-when-typically-DOWN, etc.).
Output	Structured "what's different" report: `{deviation_type, magnitude, possible_cause, urgency}`.
Why useful	Powers the edge-decay monitor. A spike in anomalies = strategy shift = demote weight.
When run	Daily cron over the latest 7 days of trades.
Cost est.	~$0.0005/trade with cache. <$1/week ongoing.

Disagreement-Narrator (Sonnet 4.6)

Input	A bucket-(ii) trade — bonereaper was right, our deterministic strategy was wrong.
Output	Structured `{missed_feature, threshold_should_be, context, weight_recommendation}`.
Why useful	This is the goldmine output. Feeds directly into weight refits — these reports become the changelog for `weights/.yaml`.
When run	Once per phase-D analysis cycle (initial study + quarterly refits).
Cost est.	~$0.01/trade with cache. ~500 bucket-(ii) trades = $5 one-time.

All three: schema-validated JSON; refusal/malformed → manual review queue. No agent decision touches a live order.

Dashboard Layout

7-tab Streamlit research console — localhost only

Built on Streamlit + Plotly + Polars + DuckDB. Reads labelled dataset directly. Auto-refreshes during dev. **Tabs 1-3 are MVP and ship in Phase G.** Tabs 4-7 are stretch goals, each gated on the previous tab shipping cleanly. This guards against dashboard scope creep eating the analysis budget.

01. Profile MVP

Total trades, rolling 7d/30d/90d win-rate with binomial CI, P&L curve, size dist, UP/DOWN balance, time-of-day heatmap.

02. Trade Explorer MVP

Sortable/filterable table; click row → reconstructed market state at T-30/-15/-10/-5s with mini spot/oracle/OB charts.

03. Disagreement Matrix

4-bucket heatmap (us_vote × their_vote × outcome). Click cells to drill into specific trades.

04. Feature Importance

Coefficient bars from logistic regression + GBM importance; per-trade SHAP-style local explanation.

05. Edge-Decay

Rolling win-rate with CIs, trade volume timeline, behavior-shift markers, signal-weight history.

06. AI Hypotheses

Pattern-Hypothesis agent narratives per trade, searchable. Anomaly-Spotter flags. Disagreement-Narrator reports.

07. Live Comparison

Activates Phase 4+. Side-by-side our decision vs bonereaper's (delayed) on the active window with both rationales.

MVP ships in Phase G (1-3); 4-7 are stretch tabs gated on each prior shipping cleanly.

Run via `make dashboard` → opens `localhost:8501`. No deployment, no auth, no external network access. Cache invalidates when underlying Parquet changes.

PHASE A · Pre-flight & Validation

0.5 day · $0

Intent: Pull bonereaper's on-chain history. Compute the abort criteria. If the wallet doesn't justify a study, stop here.

Tasks

- Set up read-only Polygon RPC access (Alchemy free tier; same as main bot Phase 2).
- Implement scripts/research/quick_audit.py: enumerate Polymarket exchange events from address, parse to (block_ts, market, side, size, fill, p
- Compute: total trades, 30/90-day rolling win-rate after fees, total volume, average size, behavior-shift dates.
- Plot quick-look matplotlib figures (no Streamlit yet) into runbooks/study/quick-audit.html.
- Document the abort criteria pass/fail in runbooks/study/preflight-decision.md.

GATE 30-day rolling win-rate ≥ 60% AND ≥500 trades observable AND no >40% volume drop in last 30 days. If any criterion fails: abort the e

Rollback: If abort: pick a different reference wallet or skip the study. Time spent: ½ day. No code in src/, only scripts/.

PHASE B · Data Ingestion

1-2 days · $0

Intent: Production-grade pipeline that captures every bonereaper trade going forward and backfills history.

Tasks

- Design doc: design/research/ingest.md — schema, retry policy, idempotency.
- Implement src/research/bonereaper_ingest.py with backfill mode (historical) and tail mode (new blocks).
- Schema for trades.parquet: (block_ts_utc, decision_ts_estimate, tx_hash, market_slug, window_ts, side, size_usd, fill_price, eventual_pnl, set
- Idempotency: re-running the backfill with the same start block produces an identical Parquet (deterministic).
- Tests: replay a fixture log, assert exact match on schema.

GATE make verify-phase-B exits 0 (mypy + ruff green; backfill produces ≥ Phase A audit count; tail mode runs 24h without gaps).

Rollback: Schema mismatch: bump schema version, write migrator. Idempotency broken: investigate; never commit non-deterministic ingestion.

PHASE C · Market-State Reconstruction

1-2 days · $0

Intent: For every bonereaper trade, rebuild the market state at T-30s/T-15s/T-10s/T-5s relative to window close, from our captured Phase 2 archiv

Tasks

- Design doc: design/research/state_replay.md — how we map block_ts → window_ts → window_open_ts → reconstruction timestamps.
- Implement src/research/state_replay.py.
- For each trade: read Phase 2 ticks/<date>/{binance,coinbase,oracle,polymarket}.parquet around window_ts ± 5 min.
- Compute snapshots: spot top-of-book + microprice (both venues), Chainlink last-answer + age, Polymarket OB top-5, our deterministic compo
- Tests: synthetic fixtures with known state, assert reconstruction matches.

GATE make verify-phase-C exits 0 (reconstruction succeeds for ≥98% of phase-B trades; failed trades logged with cause; spot/oracle timesta

Rollback: Phase-2 data gaps: extend Phase 2 capture window. Don't fake state.

PHASE D · Feature Engineering

1 day · $0

Intent: Build the labelled dataset that everything else feeds off.

Tasks

- Design doc: design/research/features.md — exact feature definitions with units and edge cases.
- Implement src/research/features.py producing labelled.parquet.
- Feature catalog: window-delta (%), oracle-vs-spot gap (%), oracle staleness (s), last-60s tick momentum, OB imbalance (top-5), time-to-close
- Outcome columns: bonereaper_side ∈ {UP, DOWN}, bonereaper_outcome ∈ {WIN, LOSE}.
- Tests: every feature has a deterministic computation and a unit test.

GATE make verify-phase-D exits 0 (mypy + ruff + tests green; labelled.parquet has ≥99% non-null feature rows).

Rollback: High null rate in a feature: that feature isn't reliably reconstructible from current data; drop it or extend Phase 2 capture.

PHASE E · Statistical Analysis

1-2 days · $0

Intent: Three analyses on the labelled dataset. No LLMs yet — pure stats and ML.

Tasks

- Behavioral fingerprint: distributions of size, time-of-day, UP/DOWN, implied conf, time-to-close at entry. Output: runbooks/study/fingerprint.md
- Feature importance: logistic regression first, gradient-boosted trees second, both predicting bonereaper_side from features. Output: runbooks/
- Disagreement matrix: for each trade, compute what our deterministic Tier-3 + ensemble would have done at the same instant; bucket (i)-(iv); c
- All three analyses reproducible from labelled.parquet via a single make analysis command.

GATE make verify-phase-E exits 0 (three reports exist; bucket-(ii) count > 0; importance ranking has at least one feature with |coef| > 2× the r

Rollback: If bucket-(ii) is empty: either we're already aligned with bonereaper (great — but suspicious), or our reconstruction is wrong (investigate).

PHASE F · AI-Narrated Insights

2-3 days · $10-50 one-time + ~$1/wk

Intent: Run the three Anthropic agents over the labelled dataset.

Tasks

- Top up Anthropic credits to ≥$50 for a clean one-time run.
- Implement src/research/agents/{pattern_hypothesis,anomaly_spotter,disagreement_narrator}.py with cached system prompts and per-trade dy
- JSON schemas committed in design/research/agent_schemas.md; pydantic validation; refusal → manual review queue (state/research/review
- Cost-tracker hard-kill at $50 one-time, $5/day ongoing.
- Output: rationale, anomaly_flag, narrator_report fields on labelled.parquet (or a sidecar table joined on tx_hash).

GATE make verify-phase-F exits 0 (≥95% of trades have valid rationale; refusal rate <5%; total spend ≤ $50).

Rollback: Refusal rate >5%: tighten prompt, retry. If still high: that's a real signal that some trades are too ambiguous — log and review.

PHASE G · Dashboard Build

2-3 days for MVP; stretch ongoing · $0

Intent: Streamlit MVP first (3 tabs). Stretch tabs gated on each prior shipping clean.

Tasks

- Design doc: design/research/dashboard.md — wireframes for all 7 tabs, MVP cut.
- Implement dashboard/research.py with tabs: 01 Profile, 02 Trade Explorer, 03 Disagreement Matrix.
- Stack: Streamlit + Plotly + Polars + DuckDB; reads labelled.parquet directly; no copies.
- make dashboard launches localhost:8501.
- Stretch tabs (in order): 04 Feature Importance, 05 Edge-Decay, 06 AI Hypotheses, 07 Live Comparison.
- Each stretch tab is its own design doc + impl + ship. Don't start tab N+1 until tab N is in use.

GATE MVP gate: bash runbooks/gates/phase-G.sh exits 0 (3 tabs render; click-through smoke test on each; no Python exceptions in logs ove

Rollback: Streamlit perf issues at scale: pre-aggregate in DuckDB. Don't compute live on every interaction.

PHASE H · Bot Integration

0.5-1 day · $0

Intent: Take the study's outputs and wire them into the main bot. Same uplift gate as the LLM ensemble: ≥3pp or it doesn't ship.

Tasks

- Generate weights/<date>.yaml from feature importance ranking (D) + disagreement-narrator recommendations (F).
- Run main bot Phase 3 backtest with new weights vs current weights as A/B.
- If uplift ≥ 3pp after fees AND no new failure mode: promote weights. Else: shelve the run, document why in runbooks/study/weight-runs/<date>
- Optional 4th ensemble agent (`behavioral_mimic`): fires only when reconstructed features strongly match bonereaper's historical entry pattern

GATE Promotion gate: A/B uplift ≥ 3pp after fees AND no regression in any kill-switch metric.

Rollback: Promotion blocked: do not ship. Shelved weights are still useful research; no harm.

PHASE I · Decay & Adversarial Monitoring

ongoing · <$5/wk

Intent: Daily monitor on bonereaper's edge. Quarterly adversarial-resistance check.

Tasks

- Daily cron: rolling 30-day win-rate + volume; trigger if win-rate < 60% sustained 14 days OR volume drop > 50% MoM. Action: demote signal w
- Weekly: re-run Anomaly-Spotter on last 7 days of trades; flagged anomaly count > 2× baseline → page Telegram.
- Quarterly: adversarial-resistance check — correlation between bonereaper entries and bait-shaped market states (deep liq gaps, extreme OB
- All monitors run on the laptop or VPS as systemd-timer units, not interactive.

GATE Monitors are running for ≥ 14 calendar days with no missed runs and no false-positive cascades.

Rollback: Monitor false-positives: tighten thresholds. False-negatives (missed obvious decay): widen lookback or add features.

Output Schemas

What every phase ships, in concrete form

trades.parquet (Phase B output)

Column	Type	Description
block_ts_utc	datetime[ns]	On-chain block timestamp, UTC.
decision_ts_estimate	datetime[ns]	block_ts minus typical confirmation latency (≈2s on Polygon).
tx_hash	str	Polygon transaction hash, primary key.
market_slug	str	btc-updown-5m-{window_ts}.
window_ts	int64	Unix-seconds window-open timestamp (divisible by 300).
side	enum	{UP, DOWN}.
size_usd	float64	Stake in USDC.
fill_price	float64	Token price paid, in [0,1].
eventual_pnl	float64	Realized after settlement (NULL until window resolves).
settled_at	datetime[ns]	Resolution timestamp, UTC.

labelled.parquet (Phase D output, joined on tx_hash)

Column group	Examples
Identity	tx_hash, window_ts, side, eventual_pnl
State T-30s	spot_binance_30, spot_coinbase_30, oracle_30, oracle_age_30, ob_top5_30, our_composite_30
State T-15s	(same fields suffixed _15)
State T-10s	(same fields suffixed _10)
State T-5s	(same fields suffixed _5)
Features	window_delta, oracle_gap, ob_imbalance, momentum_60s, ttl_s, oracle_staleness, time_of_day_bucket, implied_conf
Outcomes	bonereaper_side, bonereaper_outcome, our_side_at_decision, our_outcome_if_traded
Bucket	i_aligned_correct, ii_we_missed, iii_we_caught, iv_both_wrong
AI annotations	rationale (str), anomaly_flag (bool), narrator_report (json)

weights/.yaml (Phase H output)

Field	Description
version	ISO date + git sha of the producing study.
source	always 'bonereaper-study'.
features..weight	feature weights for the deterministic strategy.
tier3_threshold	composite confidence threshold at which Tier-3 fires.
ab_uplift_pp	measured win-rate uplift over the previous weights.
decay_signal_weight	[0,1] multiplier; reduced when bonereaper edge weakens.
notes	freeform — what changed and why; cites which disagreement-narrator reports drove the weight changes.

Risk Register

Study-specific risks and mitigations

#	Severity	Risk	Mitigation
01	BLOCKER	**Pre-flight wallet underperforms** If bonereaper's actual on-chain win-rate is below ~60%, the entire premise dies.	Phase A is a hard gate. Abort the study cleanly; pick a different wallet or skip.
02	BLOCKER	**Phase-2 data gaps** State reconstruction needs the captured archives; gaps make it impossible.	Don't start Phase C until Phase 2 of the main bot has 30+ days clean. If gaps appear: extend capture, never fake state.
03	MAJOR	**Survivorship bias** bonereaper may be the visible winner of a multi-wallet portfolio; mimicking biases toward retrospect-lucky strategies.	In Phase E, check for clustering of similar-style wallets at similar IPs/timing. Note in the report. Apply inverse-probability-weighted feature importance if multi-wallet pattern detected.
04	MAJOR	**Adversarial baiting** If bonereaper detects copiers/observers, they can plant loss-leader trades to corrupt the labelled set.	Phase I quarterly check. If correlation with bait-shaped states is high, weight strongly against mimicking those state shapes — possibly invert.
05	MAJOR	**Cost-basis mismatch** Their breakeven might differ from ours due to maker rebates, fee discounts, or co-location.	Phase D includes implied_conf (from price paid); Phase H A/B explicitly tests our cost basis, not theirs. Don't promote weights that need their cost basis to be profitable.
06	MAJOR	**Edge decay between study and live** Their strategy ages; weights from a 3-month-old study may already be stale by the time we ship.	Phase I daily monitor. Weight refits gated on live A/B at promotion. Quarterly full refit minimum.
07	MINOR	**Schema drift** Feature definitions may evolve; older labelled rows become incompatible.	Schema versioning on labelled.parquet. Migrators committed alongside any schema change.
08	MINOR	**Anthropic credit blow-up during F** Pattern-Hypothesis over a large dataset can chew $50 fast without cache discipline.	Cost-tracker hard-kill at $50 one-time. Cache the system prompt as an explicit breakpoint.
09	MINOR	**Streamlit perf at scale** 5k-trade dashboard with on-the-fly aggregation can hang the browser tab.	Pre-aggregate in DuckDB. Don't render >500 raw points in any chart; bin/sample as needed.
10	MINOR	**Reading public on-chain data — ToS** Polymarket positions are public on-chain — but always sanity-check ToS.	Quote the relevant ToS section in design/research/ingest.md before shipping the ingester. If ambiguous, escalate.

Cost & Timeline

Per-phase cumulative spend

Phase	Wall-clock	Cost	Cumulative	Driver
A — Pre-flight	0.5 day	$0	$0	—
B — Ingestion	1-2 days	$0	$0	Alchemy free tier
C — State Replay	1-2 days	$0	$0	Local compute on Phase 2 archives
D — Feature Eng.	1 day	$0	$0	Pure Python
E — Stat. Analysis	1-2 days	$0	$0	scikit-learn local
F — AI Insights	2-3 days	$10-50 one-time	$10-50	Anthropic API (cached)
G — Dashboard MVP	2-3 days	$0	$10-50	Streamlit local
H — Bot Integration	0.5-1 day	API calls in main bot Phase 3	$10-50 + main-bot Phase 3	Backtest re-run
I — Monitoring	ongoing	<$5/wk	ongoing	Daily Anomaly-Spotter calls

Total elapsed: ~10-14 working days across Phases A-G if focused. Phase H is gated on the main bot reaching its own Phase 3. Phase I runs forever as a daemon. **Total one-time cost:** $10-50 (Anthropic). **Ongoing:** <$5/week.

Operating Discipline

Same anti-vibe-coding rules apply

The 25 rules from BUILD_SPEC apply identically here. Three additions specific to a research stream:

Research stream

Every analysis is reproducible from labelled.parquet via a single make command. If you can't reproduce it tomorrow morning from a clean checkout, it doesn't ship.

Notebooks-as-source-of-truth is exactly the kind of drift this project is built to avoid.

Findings without a causal story are entertainment, not research. Every shipped weight change cites at least one Disagreement-Narrator report and one feature-importance result.

Without the story, you can't tell whether the win is real or noise — and you can't defend it next month when it stops working.

The dashboard is a tool, not a deliverable. If you spent a day on dashboard polish and didn't change weights or insights, it was a wasted day.

Streamlit is fast to prototype precisely because it's also fast to over-prototype.

Sources & References

Primary data sources, tooling, prior art

Target & Polymarket

bonereaper Polymarket profile	polymarket.com/@bonereaper
Polymarket Developer Docs	docs.polymarket.com/developers/
Polymarket Exchange contract	Polygon mainnet — see py-clob-client constants
py-clob-client (event parsing)	github.com/Polymarket/py-clob-client

Polygon / on-chain

Alchemy Polygon RPC	alchemy.com (free tier sufficient)
PolygonScan	polygonscan.com (manual verification)
Chainlink BTC/USD on Polygon	data.chain.link/feeds/polygon/mainnet/btc-usd

ML / analysis

scikit-learn (logistic, GBM)	scikit-learn.org
SHAP (feature importance)	shap.readthedocs.io
Polars (fast Parquet)	pola-rs.github.io
DuckDB (ad-hoc SQL)	duckdb.org

Dashboard

Streamlit	streamlit.io
Plotly Python	plotly.com/python/

Companion documents

Main bot master plan	/Users/hamad/Desktop/PolyMarket-5m/MASTERPLAN.md
Main bot build spec PDF	/Users/hamad/Desktop/PolyMarket-5m/BUILD_SPEC.pdf
Main bot execution runbook	/Users/hamad/Desktop/PolyMarket-5m/RUNBOOK.md
Locked decisions memory	memory/project_decisions.md

Living document. The labelled dataset and dashboard are the persistent artifacts; this PDF is the spec they implement. Refresh quarterly.